

← Back to Home

 **Zod** INTERMEDIATE

Zod

Complete reference for Zod — the TypeScript-first schema validation library with static type inference

🕒 8 min read

 zod typescript validation schema parsing type-safety forms

Share

Home > Typescript > Zod > Zod

Installation & Setup

Install Zod and start defining schemas

Install & Import

Install Zod and import it into your project

Quick Reference Detailed Example

TYPESCRIPT

 Copy

```
# === Install ===  
npm install zod  
yarn add zod
```

```

pnpm add zod
bun add zod

# === Requirements ===
# TypeScript 4.5+
# tsconfig.json must have "strict": true (recommended)

// === Import styles ===
import * as z from "zod";           // Recommended
import { z } from "zod";           // Also works

// === Basic usage ===
const UserSchema = z.object({
  name: z.string(),
  age: z.number(),
});

// Parse data
const user = UserSchema.parse({ name: "Alice", age: 30 });

// Infer TypeScript type
type User = z.infer<typeof UserSchema>;
// => { name: string; age: number }

```

- 💡 Enable "strict": true in tsconfig.json for full type inference benefits
- ⚡ Zod has zero dependencies and works in Node, Bun, Deno, and the browser
- ✳️ Use import * as z for the recommended namespace import pattern
- 🟢 One schema gives you both runtime validation AND a TypeScript type

install setup

Primitive Schemas

Built-in schemas for JavaScript primitive types

Primitive Types

Schemas for strings, numbers, booleans, dates, and special types

[Quick Reference](#)

[Detailed Example](#)

TYPESCRIPT

 Copy

```
import * as z from "zod";

// === Primitives ===
const stringSchema = z.string();
const numberSchema = z.number();
const booleanSchema = z.boolean();
const dateSchema = z.date();
const bigintSchema = z.bigint();
const symbolSchema = z.symbol();

// === Special types ===
z.null();           // null
z.undefined();      // undefined
z.void();           // void return types
z.any();            // any value
z.unknown();        // unknown value (safer than any)
z.never();          // never matches
z.literal("foo");   // exact value match

// === NaN ===
z.nan();            // matches NaN

// === Parse examples ===
stringSchema.parse("hello");    // "hello"
numberSchema.parse(42);         // 42
booleanSchema.parse(true);      // true
dateSchema.parse(new Date());   // Date object

// === Throws on invalid input ===
try {
  stringSchema.parse(123);
} catch (err) {
```

```
if (err instanceof z.ZodError) {  
  console.log(err.issues);  
}  
}
```

- 💡 Zod schemas are immutable — methods return new schemas instead of mutating
- ⚡ Use `z.unknown()` instead of `z.any()` to keep type safety throughout your code
- 📌 `z.literal()` matches an exact value — useful for tagged unions and constants
- 🟢 `z.date()` validates Date objects, not date strings — use `z.coerce.date()` for strings

primitives types

String Validation

String-specific validators and format checks

String Validators

Length, format, and pattern validators for strings

[Quick Reference](#) [Detailed Example](#)

TYPESCRIPT

 Copy

```
// === Length validators ===  
z.string().min(3, "Too short");  
z.string().max(20, "Too long");  
z.string().length(10);  
z.string().nonempty();           // min(1)  
  
// === Format validators (top-level in Zod 4) ===
```

```
z.email();
z.url();
z.uuid();
z.uuidv4();
z.uuidv7();
z.cuid();
z.cuid2();
z.ulid();
z.nanoid();
z.ipv4();
z.ipv6();
z.cidrv4();
z.iso.date();           // ISO date "2024-01-01"
z.iso.time();           // "13:45:30"
z.iso.datetime();       // ISO datetime
z.iso.duration();       // ISO 8601 duration
z.base64();
z.jwt();
z.hex();
z.emoji();

// === Pattern matching ===
z.string().regex(/^[a-z]+$/, "Lowercase letters only");
z.string().startsWith("https://");
z.string().endsWith(".com");
z.string().includes("@");

// === Transformations ===
z.string().trim();
z.string().toLowerCase();
z.string().toUpperCase();
z.string().normalize();  // Unicode normalization

// === Combine ===
const UsernameSchema = z.string()
  .min(3, "Minimum 3 characters")
  .max(20, "Maximum 20 characters")
  .regex(/^[a-z0-9_]+$/, "Lowercase letters, numbers, underscore")
  .toLowerCase();
```

💡 In Zod 4, format validators like `z.email()` are top-level functions, not `z.string().email()`

⚡ Chain string methods to build complex validators — they all return `ZodString`

📌 Use `z.iso.datetime()` for ISO 8601 strings and `z.coerce.date()` to convert to `Date` objects

● String transformations like `.trim()` and `.toLowerCase()` apply during parsing

string validation email url

Number Validation

Number-specific validators for ranges and integer constraints

Number Validators

Range, integer, and sign validators for numbers

Quick Reference

[Detailed Example](#)

TYPESCRIPT

📋 Copy

```
// === Range validators ===
z.number().min(0, "Must be at least 0");
z.number().max(100, "Must be at most 100");
z.number().gt(0);    // Greater than (exclusive)
z.number().gte(0);   // Greater than or equal (same as min)
z.number().lt(100);  // Less than (exclusive)
z.number().lte(100); // Less than or equal (same as max)

// === Integer constraints ===
z.number().int();           // Must be integer
z.number().int().min(1);    // Positive integer

// === Sign ===
```

```

z.number().positive();           // > 0
z.number().nonnegative();        // >= 0
z.number().negative();           // < 0
z.number().nonpositive();        // <= 0

// === Other ===
z.number().multipleOf(5);        // Divisible by 5
z.number().step(0.01);           // Same as multipleOf
z.number().finite();             // Not Infinity
z.number().safe();               // Within Number.MAX_SAFE_INTEGER

// === BigInt has same validators ===
z.bigint().min(0n).max(1000n);

// === Practical example ===
const PriceSchema = z.number()
  .positive("Price must be positive")
  .multipleOf(0.01, "Price must have at most 2 decimals")
  .max(99999, "Price too high");

```

- 💡 `z.number()` rejects NaN by default — use `z.nan()` if you need to allow it
- ⚡ Use `.int()` before other validators for cleaner integer-only schemas
- ✳️ `.multipleOf()` works for floats like 0.01 — perfect for currency validation
- 🟢 `.safe()` ensures the number fits in JavaScript safe integer range

number validation integer

Object Schemas

Define and manipulate object shapes with `z.object()`

Defining Objects

Create object schemas with required and optional properties

[Quick Reference](#)

[Detailed Example](#)

TYPESCRIPT

 Copy

```
// === Basic object schema ===
const UserSchema = z.object({
  id: z.uuid(),
  name: z.string().min(1),
  email: z.email(),
  age: z.number().int().positive().optional(),
  isAdmin: z.boolean().default(false),
  createdAt: z.date(),
});

// === Infer TypeScript type ===
type User = z.infer<typeof UserSchema>;
// {
//   id: string;
//   name: string;
//   email: string;
//   age?: number;
//   isAdmin: boolean;
//   createdAt: Date;
// }

// === Parse data ===
const user = UserSchema.parse({
  id: crypto.randomUUID(),
  name: "Alice",
  email: "alice@example.com",
  createdAt: new Date(),
});

// === Unknown key handling ===
// Default: strip unknown keys
z.object({ name: z.string() }).parse({ name: "Alice", extra: 1 })
// => { name: "Alice" }

// strictObject: throw on unknown keys
```

```
z.strictObject({ name: z.string() }).parse({ name: "Alice", ext
// => throws
```

```
// looseObject: pass through unknown keys
z.looseObject({ name: z.string() }).parse({ name: "Alice", extr
// => { name: "Alice", extra: 1 }
```

```
// catchall: validate unknown keys with another schema
const Config = z.object({ name: z.string() }).catchall(z.number
Config.parse({ name: "app", port: 3000, version: 1 }); // OK
```

- 💡 `z.object()` strips unknown keys by default — use `strictObject` to reject them
- ⚡ `z.infer` extracts a TypeScript type from a schema — single source of truth
- ✳️ Optional properties become `T | undefined` in the inferred type
- 🟢 Use `catchall()` to validate dynamic keys against a fallback schema

object schema

Object Methods

Transform existing object schemas with `extend`, `pick`, `omit`, `partial`

[Quick Reference](#) [Detailed Example](#)

TYPESCRIPT

 Copy

```
const UserSchema = z.object({
  id: z.uuid(),
  name: z.string(),
  email: z.email(),
  password: z.string().min(8),
});
```

```
// === pick: select specific keys ===
```

```
const PublicUser = UserSchema.pick({
  id: true,
  name: true,
});
// { id: string; name: string }

// === omit: exclude specific keys ===
const SafeUser = UserSchema.omit({
  password: true,
});
// { id, name, email }

// === partial: make all keys optional ===
const UserUpdate = UserSchema.partial();
// { id?, name?, email?, password? }

// Partial with specific keys
const PartialUpdate = UserSchema.partial({
  email: true,
  password: true,
});

// === required: make optional keys required ===
const RequiredUser = UserSchema.required();

// === extend: add new keys ===
const AdminUser = UserSchema.extend({
  role: z.enum(["admin", "superadmin"]),
  permissions: z.array(z.string()),
});

// === merge two schemas (Zod 3 / use extend in Zod 4) ===
const Timestamps = z.object({
  createdAt: z.date(),
  updatedAt: z.date(),
});
const UserWithTimestamps = UserSchema.extend(Timestamps.shape);

// === Access shape ===
UserSchema.shape.email; // ZodString
UserSchema.keyof();      // ZodEnum of keys
```

- 💡 `.pick()` and `.omit()` take an object with true values, not an array of keys
- ⚡ `.partial()` is perfect for PATCH update endpoints where any field is optional
- ✳️ `.extend()` takes priority over the original — duplicate keys are overridden
- 🟢 Use `.keyof()` to get an enum of all valid keys for type-safe field references

object pick omit extend

Arrays, Tuples, Sets & Maps

Schemas for collection types

Collection Schemas

Define arrays, tuples, sets, maps, and records

[Quick Reference](#)

[Detailed Example](#)

TYPESCRIPT

 Copy

```
// === Array ===
const StringArray = z.array(z.string());
// or: z.string().array()

StringArray.parse(["a", "b", "c"]); // OK

// Array validators
z.array(z.string()).min(1, "At least one item");
z.array(z.string()).max(10);
z.array(z.string()).length(5);
z.array(z.string()).nonempty();

// === Tuple (fixed length) ===
const Coord = z.tuple([z.number(), z.number()]);
Coord.parse([10, 20]); // [10, 20]
```

```

// Tuple with rest
const NamedCoords = z.tuple([z.string()])
  .rest(z.number());
NamedCoords.parse(["origin", 0, 0, 0]);

// === Record (object with dynamic keys) ===
const Scores = z.record(z.string(), z.number());
Scores.parse({ alice: 100, bob: 85 });

// Record with enum keys (exhaustive)
const Status = z.record(
  z.enum(["pending", "active", "done"]),
  z.string()
);

// Partial record (keys optional)
const PartialStatus = z.partialRecord(
  z.enum(["pending", "active", "done"]),
  z.string()
);

// === Map ===
const UserMap = z.map(z.string(), z.object({ name: z.string() }
const m = new Map();
m.set("u1", { name: "Alice" });
UserMap.parse(m);

// === Set ===
const UniqueIds = z.set(z.string()).min(1).max(100);
UniqueIds.parse(new Set(["a", "b", "c"]));

```

- 💡 `z.array(z.X)` and `z.X.array()` are equivalent — pick whichever reads better
- ⚡ Use `z.tuple()` for fixed-length arrays with positional types like coordinates
- ✳️ `z.record()` validates plain objects with dynamic keys — different from `z.map()`
- `z.set()` and `z.map()` work on actual `Set/Map` instances, not arrays/objects

Unions, Intersections & Discriminated Unions

Combine multiple schemas with unions and intersections

Combining Schemas

Union, intersection, and discriminated union patterns

[Quick Reference](#)

[Detailed Example](#)

TYPESCRIPT

 Copy

```
// === Union (any of these types) ===
const StringOrNumber = z.union([z.string(), z.number()]);
// or fluent: z.string().or(z.number())

StringOrNumber.parse("hello"); // OK
StringOrNumber.parse(42);      // OK
StringOrNumber.parse(true);    // throws

// === Discriminated union (recommended for tagged types) ===
// Faster validation and clearer error messages
const Shape = z.discriminatedUnion("kind", [
  z.object({
    kind: z.literal("circle"),
    radius: z.number().positive(),
  }),
  z.object({
    kind: z.literal("square"),
    size: z.number().positive(),
  }),
  z.object({
    kind: z.literal("rectangle"),
    width: z.number().positive(),
  })
]);
```

```

        height: z.number().positive(),
    })),
]);

type Shape = z.infer<typeof Shape>;
// { kind: "circle"; radius: number }
// | { kind: "square"; size: number }
// | { kind: "rectangle"; width: number; height: number }

const shape = Shape.parse({ kind: "circle", radius: 10 });
if (shape.kind === "circle") {
    console.log(shape.radius); // TypeScript narrows the type
}

// === Intersection (all properties combined) ===
const Person = z.object({ name: z.string() });
const Employee = z.object({ company: z.string() });
const PersonEmployee = z.intersection(Person, Employee);
// or: Person.and(Employee)
// { name: string; company: string }

// Prefer .extend() for object schemas – clearer intent
const PersonEmployee2 = Person.extend({ company: z.string() });

```

- 💡 Use discriminatedUnion when objects share a common literal field — much faster than union
- ⚡ Discriminated unions give clearer error messages by checking only the matching variant
- ✳️ Prefer .extend() over intersection for combining object schemas — same result, cleaner
- z.union() tries each schema in order — first match wins

union intersection discriminated

Enums & Literals

Enum & Literal Schemas

Restrict values to a known set with `z.enum()` and `z.literal()`

[Quick Reference](#)

[Detailed Example](#)

TYPESCRIPT

 Copy

```
// === Literal (single exact value) ===
const Tuna = z.literal("tuna");
Tuna.parse("tuna"); // OK
Tuna.parse("salmon"); // throws

// Multiple literals = use enum
const Fish = z.literal(["tuna", "salmon", "trout"]); // Zod 4

// === Enum (string set) ===
const FishEnum = z.enum(["tuna", "salmon", "trout"]);
type Fish = z.infer<typeof FishEnum>;
// "tuna" | "salmon" | "trout"

FishEnum.parse("tuna"); // OK

// Access enum values
FishEnum.options; // ["tuna", "salmon", "trout"]
FishEnum.enum.tuna; // "tuna"

// Enum methods
FishEnum.exclude(["trout"]); // ["tuna", "salmon"]
FishEnum.extract(["tuna"]); // ["tuna"]

// === Native enum (TypeScript enum) ===
enum Status {
  ACTIVE = "ACTIVE",
  INACTIVE = "INACTIVE",
  PENDING = "PENDING",
}
```

```
const StatusSchema = z.nativeEnum(Status);
StatusSchema.parse("ACTIVE"); // OK
StatusSchema.parse(Status.ACTIVE); // OK

// === As const arrays ===
const ROLES = ["admin", "user", "guest"] as const;
const RoleSchema = z.enum(ROLES);
```

- 💡 `z.enum()` takes a tuple of strings — much better DX than `z.nativeEnum()`
- ⚡ Use as `const` with `z.enum()` to share the values as both runtime and types
- 📌 `z.literal()` is perfect for discriminator fields in discriminated unions
- 🟢 Access `.options` on a `z.enum` to get the array of valid values at runtime

enum literal

Optional, Nullable, Default & Catch

Handle missing values, defaults, and fallbacks

Handling Missing Values

Optional, nullable, default values, and catch fallbacks

[Quick Reference](#)

[Detailed Example](#)

TYPESCRIPT

 Copy

```
// === Optional (allows undefined) ===
const OptionalString = z.string().optional();
// or: z.optional(z.string())
```

```
OptionalString.parse("hello");    // "hello"
OptionalString.parse(undefined);  // undefined
OptionalString.parse(null);       // throws

// === Nullable (allows null) ===
const NullableString = z.string().nullable();

NullableString.parse("hello");    // "hello"
NullableString.parse(null);       // null
NullableString.parse(undefined);  // throws

// === Nullish (allows both null and undefined) ===
const NullishString = z.string().nullish();
NullishString.parse("hello");     // "hello"
NullishString.parse(null);        // null
NullishString.parse(undefined);   // undefined

// === Default (applied when undefined) ===
const Theme = z.string().default("light");
Theme.parse(undefined);           // "light"
Theme.parse("dark");              // "dark"

// Default with function (called per parse)
const Id = z.string().default(() => crypto.randomUUID());

// === Catch (fallback for any error) ===
const Port = z.number().catch(3000);
Port.parse("invalid");            // 3000 (no error)
Port.parse(8080);                 // 8080

// Catch with function (gets the error)
const SafeNumber = z.number().catch((ctx) => {
  console.log(ctx.error.issues);
  return 0;
});

// === In objects ===
const ConfigSchema = z.object({
  name: z.string(),
  port: z.number().default(3000),
  host: z.string().default("localhost"),
  debug: z.boolean().optional(),
});
```

```
apiKey: z.string().nullable(),
});
```

- 💡 `.default()` only kicks in when the value is undefined, not null
- ⚡ `.catch()` is like a try/catch — it swallows ALL errors and returns the fallback
- 📌 Use `.nullish()` for fields that may be null OR undefined (common in databases)
- 🟢 `.default()` with a function generates a fresh value on every parse — ideal for IDs

optional nullable default

Parsing & Error Handling

Parse data and handle validation errors

parse() vs safeParse()

Throwing and non-throwing parse methods

[Quick Reference](#)

[Detailed Example](#)

TYPESCRIPT

 Copy

```
const UserSchema = z.object({
  name: z.string(),
  age: z.number(),
});

// === parse() - throws ZodError on failure ===
try {
  const user = UserSchema.parse({ name: "Alice", age: 30 });
  // user is fully typed
} catch (err) {
```

```

    if (err instanceof z.ZodError) {
      console.log(err.issues);
      // [
      //   {
      //     code: 'invalid_type',
      //     expected: 'number',
      //     path: ['age'],
      //     message: 'Invalid input: expected number'
      //   }
      // ]
    }
  }

  // === safeParse() – returns discriminated union ===
  const result = UserSchema.safeParse({ name: "Bob", age: "30" })

  if (result.success) {
    console.log(result.data); // typed user
  } else {
    console.log(result.error.issues);
    // Access first issue
    console.log(result.error.issues[0].message);
  }

  // === Async parsing (for async refinements) ===
  const user = await UserSchema.parseAsync(data);
  const result = await UserSchema.safeParseAsync(data);

  // === Format errors for display ===
  const result = UserSchema.safeParse(data);
  if (!result.success) {
    // Flatten errors by field
    const flat = z.flattenError(result.error);
    // { formErrors: [], fieldErrors: { age: ["Invalid input: exp

    // Tree-shaped errors
    const tree = z.treeifyError(result.error);

    // Pretty-printed
    const pretty = z.prettifyError(result.error);
  }

```

- 💡 Prefer `safeParse()` in API routes — no `try/catch`, cleaner control flow
- ⚡ `z.flattenError()` is perfect for form validation — gives field-level error arrays
- ✳️ Use `parseAsync()` when your schema has async refinements like database lookups
- 🟢 `ZodError.issues` is an array — each issue has `code`, `path`, `message` for granular handling

[parse](#) [safeParse](#) [errors](#)

Customizing Error Messages

Set custom error messages on schemas and validators

[Quick Reference](#) [Detailed Example](#)

TYPESCRIPT

 Copy

```
// === Top-level error message ===
const NameSchema = z.string({
  message: "Name is required and must be a string",
});

// === Per-validator messages ===
const Username = z.string()
  .min(3, "Username must be at least 3 characters")
  .max(20, "Username must be at most 20 characters")
  .regex(/^[a-z0-9_]+$/, "Only lowercase letters, numbers, and

// === Object field messages ===
const SignupSchema = z.object({
  email: z.email({ message: "Please enter a valid email" }),
  password: z.string({ message: "Password is required" })
    .min(8, "Password must be at least 8 characters")
    .regex(/[A-Z]/, "Must contain an uppercase letter")
    .regex(/[0-9]/, "Must contain a number"),
  age: z.number({ message: "Age must be a number" })
```

```

    .int("Age must be a whole number")
    .min(13, "You must be at least 13 years old"),
  });

// === Function-based messages (access value/context) ===
const PasswordSchema = z.string().min(8, (ctx) => {
  return `Password too short. Got ${ctx.value.length} character`
});

// === Global error customization ===
import { z } from "zod";

z.config({
  customError: (issue) => {
    if (issue.code === "invalid_type") {
      return `Expected ${issue.expected}, got ${typeof issue.in
    }
  },
});

```

- 💡 In Zod 4, error messages use the { message: "..." } params object pattern
- ⚡ Use function-based messages to access the invalid value in your error string
- ✳️ z.config() sets a global error handler — useful for i18n and consistent messaging
- 🟢 Per-validator messages are more specific than top-level — order matters

errors messages validation

Refinements & Transforms

Custom validation logic and data transformations

refine() & superRefine()

Add custom validation logic to any schema

[Quick Reference](#)[Detailed Example](#)

TYPESCRIPT

 Copy

```
// === Simple refinement ===
const PositiveInt = z.number().refine(
  (val) => Number.isInteger(val) && val > 0,
  { message: "Must be a positive integer" }
);

PositiveInt.parse(5);    // OK
PositiveInt.parse(-1);   // throws
PositiveInt.parse(1.5);  // throws

// === Object-level refinement (cross-field) ===
const SignupSchema = z.object({
  password: z.string().min(8),
  confirm: z.string(),
}).refine((data) => data.password === data.confirm, {
  message: "Passwords don't match",
  path: ["confirm"], // attach error to confirm field
});

// === Async refinement (database checks, etc.) ===
const UniqueEmailSchema = z.email().refine(
  async (email) => {
    const exists = await db.user.findUnique({ where: { email } })
    return !exists;
  },
  { message: "Email already registered" }
);

// Must use parseAsync with async refinements
await UniqueEmailSchema.parseAsync("new@example.com");

// === superRefine for multiple issues ===
const ListSchema = z.array(z.string()).superRefine((arr, ctx) => {
  if (arr.length < 2) {
    ctx.addIssue({
```

```

        code: "too_small",
        minimum: 2,
        type: "array",
        inclusive: true,
        message: "Need at least 2 items",
    });
}
if (new Set(arr).size !== arr.length) {
    ctx.addIssue({
        code: "custom",
        message: "No duplicates allowed",
    });
}
});

// === .check() – Zod 4 alternative ===
const Username = z.string().check((ctx) => {
    if (ctx.value.length < 3) {
        ctx.issues.push({
            code: "too_small",
            input: ctx.value,
            minimum: 3,
            origin: "string",
        });
    }
});

```

- 💡 Use `refine()` for single conditions, `superRefine()` when you need multiple errors
- ⚡ Cross-field validation (like password confirm) requires object-level `refine()`
- 📍 Set `{ path: ["field"] }` to attach the error to a specific field instead of the root
- 🟢 Async refinements MUST be used with `parseAsync` — sync parse will throw

refine validation custom

transform(), pipe() & coerce

Convert and transform data during validation

[Quick Reference](#)

[Detailed Example](#)

TYPESCRIPT

 Copy

```
// === transform: convert validated data ===
const UpperString = z.string().transform((val) => val.toUpperCase())
UpperString.parse("hello"); // "HELLO"

// Chain transforms
const NormalizedEmail = z.email()
  .transform((email) => email.toLowerCase())
  .transform((email) => email.trim());

// === pipe: chain validation + transformation ===
const StringToInt = z.string()
  .transform((val) => parseInt(val, 10))
  .pipe(z.number().int().positive());

StringToInt.parse("42"); // 42
StringToInt.parse("-5"); // throws (not positive)
StringToInt.parse("abc"); // throws (NaN)

// === preprocess: transform BEFORE validation ===
const FlexibleNumber = z.preprocess(
  (val) => (typeof val === "string" ? parseInt(val, 10) : val),
  z.number()
);
FlexibleNumber.parse("100"); // 100
FlexibleNumber.parse(100); // 100

// === coerce: shorthand for type coercion ===
z.coerce.string().parse(123); // "123"
z.coerce.number().parse("42"); // 42
z.coerce.boolean().parse("true"); // true
z.coerce.date().parse("2024-01-01"); // Date object
z.coerce.bigint().parse("100"); // 100n

// === Practical: form data parsing ===
const QuerySchema = z.object({
```

```

    page: z.coerce.number().int().positive().default(1),
    limit: z.coerce.number().int().min(1).max(100).default(20),
    search: z.string().optional(),
  });

  // Parse URL search params
  const params = QuerySchema.parse({
    page: "2",          // string from URL
    limit: "50",
    search: "zod",
  });
  // { page: 2, limit: 50, search: "zod" }

  // === Input vs Output types ===
  const Schema = z.string().transform((s) => s.length);
  type Input = z.input<typeof Schema>;    // string
  type Output = z.output<typeof Schema>;  // number

```

- 💡 `z.coerce` is a shortcut for the most common preprocess pattern — type coercion
- ⚡ Use `coerce` for query params and form data where everything starts as strings
- 🔗 `.pipe()` chains a transform into another schema for additional validation
- 🟢 Input and output types differ when transforms change the type — use `z.input/z.output`

transform coerce pipe

Type Inference

Extract TypeScript types from Zod schemas

`z.infer`, `z.input` & `z.output`

Use schemas as the single source of truth for runtime and types

TYPESCRIPT

 Copy

```
// === z.infer – same as z.output ===
const UserSchema = z.object({
  id: z.uuid(),
  name: z.string(),
  email: z.email(),
  age: z.number().optional(),
});

type User = z.infer<typeof UserSchema>;
// {
//   id: string;
//   name: string;
//   email: string;
//   age?: number;
// }

// === z.input vs z.output (different with transforms) ===
const StringToNumber = z.string().transform((s) => parseInt(s))

type Input = z.input<typeof StringToNumber>; // string
type Output = z.output<typeof StringToNumber>; // number

// === Use inferred types in functions ===
function createUser(data: z.input<typeof UserSchema>): User {
  return UserSchema.parse(data);
}

// === API request/response types ===
const CreateUserRequest = z.object({
  name: z.string(),
  email: z.email(),
});

type CreateUserRequest = z.infer<typeof CreateUserRequest>;

// In an API handler
async function POST(req: Request) {
```

```

const body = await req.json();
const result = CreateUserRequest.safeParse(body);

if (!result.success) {
  return Response.json({ errors: result.error.issues }, { sta
}

// result.data is fully typed
const user = await db.user.create({ data: result.data });
return Response.json(user);
}

// === Schema as form values ===
import { useForm } from "react-hook-form";
import { zodResolver } from "@hookform/resolvers/zod";

const FormSchema = z.object({
  email: z.email(),
  password: z.string().min(8),
});

type FormValues = z.infer<typeof FormSchema>;

const form = useForm<FormValues>({
  resolver: zodResolver(FormSchema),
});

```

- 💡 `z.infer` gives you a TypeScript type for free — no need to write interfaces twice
- ⚡ Use `z.input` for function parameters and `z.output` for return types when transforms are involved
- ✳️ Schemas are the single source of truth — change once, types update everywhere
- Pair Zod with `@hookform/resolvers/zod` for fully typed React Hook Form validation

infer types inference